

Layered Prompt Protection for AI Agents in Customer-Controlled Cloud Accounts: Model Robustness and Defense-Layer Contribution

Protección en capas de prompts para agentes de IA en cuentas de nube controladas por el cliente: robustez frente al modelo y contribución de las capas de defensa

David Petrocelli¹  and Juan Manuel Fernández^{1,2} 

¹Universidad Nacional de Luján (UNLu), Buenos Aires, Argentina

dpetrocelli@unlu.edu.ar, jmfernandez@unlu.edu.ar

²Buenos Aires Institute of Technology (ITBA), Argentina

jmfernandez@itba.edu.ar

Abstract

Large language models (LLMs) relocate a substantial share of application logic from executable code to natural-language system prompts. When AI agents must run inside customer-controlled cloud accounts to satisfy data residency constraints, these prompts become exposed to privileged principals even though they constitute proprietary intellectual property. This article treats the problem as a cross-domain trust problem and presents a defense-in-depth architecture that composes four software-based layers, cryptographic prompt delivery, OWASP-aligned input validation, output filtering with steganographic watermarking, and cross-account monitoring, into a trust chain that provides prompt confidentiality, output integrity, and forensic traceability without Trusted Execution Environments. The architecture is implemented with standard serverless services and validated through penetration testing and a live two-account deployment. This article extends our previous conference work with two experimental contributions: a multi-model evaluation that measures the robustness of the defense layers across nine LLMs from six model families under an identical protocol, and an ablation study that quantifies the individual and complementary contribution of each defense layer. **[TODO: Add the main quantitative findings once the experimental results are available.]**

Keywords: AI security, defense-in-depth, multi-tenant cloud, prompt protection, watermarking

Resumen

Los modelos de lenguaje de gran tamaño (LLM) trasladan una parte sustancial de la lógica de las aplicaciones desde el código ejecutable hacia *system prompts* en lenguaje natural. Cuando los agentes de IA deben ejecutarse dentro de cuentas de nube controladas por el

cliente para cumplir requisitos de residencia de datos, esos prompts quedan expuestos a principales con privilegios, aun cuando constituyen propiedad intelectual. Este artículo aborda el problema como un problema de confianza entre dominios y presenta una arquitectura de defensa en profundidad que compone cuatro capas de software, entrega criptográfica del prompt, validación de entradas alineada con OWASP, filtrado de salidas con marca de agua esteganográfica y monitoreo entre cuentas, en una cadena de confianza que provee confidencialidad del prompt, integridad de las salidas y trazabilidad forense sin entornos de ejecución confiables. La arquitectura se implementa con servicios *serverless* estándar y se valida mediante pruebas de penetración y un despliegue real en dos cuentas. Este artículo extiende nuestro trabajo previo de conferencia con dos contribuciones experimentales: una evaluación multimodelo que mide la robustez de las capas de defensa sobre nueve LLM de seis familias bajo un protocolo idéntico, y un estudio de ablación que cuantifica la contribución individual y complementaria de cada capa. **[TODO: Incorporar los hallazgos cuantitativos principales cuando estén disponibles los resultados experimentales.]**

Palabras claves: defensa en profundidad, marca de agua, nube multiinquilino, protección de prompts, seguridad de IA

1 Introduction

Over the last few years, large language models (LLMs) have changed the way software systems encode and operationalize knowledge [1, 2]. Conventional systems concentrate their intellectual value in executable code, where behavior is implemented explicitly. AI agents, by contrast, are increasingly governed by *system prompts*: natural-language specifications of behavior, domain expertise, and safety constraints that act as the primary control interface and externalize

system logic into interpretable text [3, 4, 5]. A considerable share of the logic that used to reside in compiled artifacts therefore now lives in natural-language specifications. This displacement creates new challenges for intellectual property protection, and their practical relevance grows with the *AI agent as a service* market [6].

The problem is most acute in multi-tenant cloud environments, where deployment and data ownership belong to different administrative domains. AI agents are frequently required to execute inside the customer’s own cloud account in order to satisfy data residency constraints and regulatory frameworks such as the General Data Protection Regulation (GDPR), the Health Insurance Portability and Accountability Act (HIPAA), and Service Organization Control 2 (SOC 2) [7]. At the same time, the system prompt must be treated as proprietary knowledge, because it encodes the differentiated capabilities on which the commercial value of the service rests. The two requirements conflict: running the agent in a customer-controlled environment opens an exposure surface in which prompt configurations become readable by privileged actors, which defeats their confidentiality [8, 9].

The conflict is practical rather than hypothetical. Managed agent platforms such as Amazon Bedrock Agents, Azure AI Agent Service, and Vertex AI Agents keep agent configurations in account-level resources that administrative principals can usually read. Prompt extraction has been demonstrated without any privileged access [10, 11], and prompt injection [12, 13], ranked first in the Open Worldwide Application Security Project (OWASP) LLM Top-10, enlarges the attack surface further by enabling indirect manipulation of model behavior.

Existing defenses cover this problem only partially and, as a rule, in isolation. Cryptographic mechanisms protect prompts in transit or under controlled access [14]. Software defenses address individual vectors through filtering, obfuscation, or leakage detection [15, 16, 17, 18, 19]. Hardware-based approaches rely on Trusted Execution Environments (TEEs), which offer strong isolation but require dedicated hardware, add measurable overhead [20], and provide no support for forensic attribution [21, 22]. Surveys of agentic security [9] document the breadth of the threat landscape, but they report no deployed architecture that protects prompts systematically across administrative domains.

We therefore frame prompt protection in multi-tenant LLM systems as a cross-domain trust problem rather than as the design of one more isolated security mechanism [23, 24, 9]. This framing calls for reasoning about how trust is split between vendor and customer environments, how adversaries exploit the resulting trust boundary, and how several defensive mechanisms can be composed so that the security guarantees survive the compromise of any single one.

This article presents a defense-in-depth architecture that composes standard cloud services into a four-layer trust chain: cryptographic prompt delivery (Layer 1), OWASP-aligned input validation (Layer 2), output filtering with steganographic watermarking (Layer 3), and cross-account monitoring (Layer 4). The composition provides prompt confidentiality, output integrity, and forensic traceability while preserving deployment flexibility in multi-tenant settings. Since it depends on no specialized hardware and distributes the defenses across independent layers, the architecture is portable across cloud providers and tolerant to the failure of individual layers.

A layered design, however, raises two questions that a single deployment cannot answer. Layers 2 and 3 interact with the behavior of the underlying LLM, so their effectiveness may vary across model families; and the value of composing several layers rests on the assumption that they are complementary, which has to be shown empirically. We formulate these questions as follows.

- **RQ1 (model robustness).** How robust is the proposed defense-in-depth architecture across different LLM families?
- **RQ2 (layer contribution).** How much does each defense layer contribute to the overall security effectiveness of the architecture?

This article is an extended and revised version of our work presented at JCC-BD&ET 2026 [25], which introduced the threat model, the four-layer architecture, the serverless reference implementation, and its validation through controlled penetration testing and a live two-account deployment. The present version extends that work with the following contributions: (i) a systematic multi-model evaluation that applies an identical experimental protocol to nine LLMs from six model families, both proprietary and open-weight, in order to answer RQ1; (ii) a defense-layer ablation study that quantifies the individual and complementary contribution of the defense layers, including the cases in which an attack that passes Layer 2 is intercepted by Layer 3, in order to answer RQ2; and (iii) an expanded experimental methodology, a more detailed account of the reference implementation and of its reproducibility environment, and a discussion informed by the new evaluations. The sections shared with the conference version have been revised and rewritten.

The evaluation covers three complementary dimensions: (i) security effectiveness, understood as resistance to prompt extraction and adversarial manipulation; (ii) performance overhead, measured as the latency added by the defense layers; and (iii) practical deployability, including portability across providers and feasibility under realistic operational constraints.

The remainder of this article is organized as follows. Section 2 reviews related work. Section 3 formalizes

the threat model and the security requirements. Section 4 describes the architecture, and Section 5 its reference implementation and reproducibility environment. Section 6 presents the experimental methodology, Section 7 the security evaluation, and Section 8 the performance and cost evaluation. Section 9 discusses the results and their limitations, and Section 10 concludes the article.

2 Related Work

Prompt extraction and manipulation attacks against LLM applications have received considerable attention. PLeak [10] casts prompt extraction as an optimization problem and reconstructs system prompts almost completely from crafted queries. Multi-turn leakage attacks [11] exploit the sycophancy of instruction-tuned models to recover hidden instructions incrementally, with success rates that reach 86.2%. Systematic reviews of prompt injection [12] organize direct, indirect, and composed vectors and show that keeping the prompt confidential is not sufficient to guarantee secure behavior.

Software-based defenses respond to these threats at specific points of the interaction pipeline. ProxyPrompt [15] interposes a proxy that shields the real prompt from extraction, and SysVec [16] replaces the textual prompt with a non-invertible vector representation. Other proposals concentrate on validation and filtering: PromptGuard [18] applies structured filtering against injection, Encrypted Prompt [17] gates prompt access through permission-bearing tokens, and PromptKeeper [19] frames leakage detection as hypothesis testing. Attention Tracker [26] inspects model internals to recognize attention patterns typical of adversarial inputs. These mechanisms are effective within their scope, but each operates inside a single trust boundary and targets one attack vector at a time.

A second line of work composes several defenses to improve robustness. TRYLOCK [27] stacks jailbreak defenses to lower attack success rates, and Counter-mind [28] adds auditing and monitoring capabilities to the inference pipeline. Analyses of serverless machine learning security [29] describe the enlarged attack surface of cloud-native deployments. These contributions support the value of layered protection, yet they concentrate on inference-time robustness and leave cross-domain deployment unaddressed.

At the infrastructure level, TEE-based systems such as Petridish [21] and Omega [22] run inference inside secure enclaves so that neither inputs nor prompts are directly accessible. Recent measurements report moderate overheads under controlled conditions [20]. Nevertheless, these systems depend on specific hardware, constrain deployment options, and offer no built-in support for forensic attribution or cross-system monitoring.

Watermarking addresses the complementary prob-

lem of output provenance. Token-level schemes [30] were followed by scalable, production-oriented frameworks such as Waterfall [31] and SynthID [32]. Zero-width character (ZWC) steganography [33] embeds covert signals in generated text, although work on steganographic collusion [34] shows that covert channels can also be abused. All of these techniques target output integrity and attribution, not the confidentiality of system prompts.

Taken together, the literature treats prompt confidentiality, input validation, output filtering, and attribution as separate problems, usually inside one trust boundary. The separation between deployment and ownership that characterizes multi-tenant settings adds constraints that these isolated solutions do not consider. A further gap concerns evidence: proposals that combine several mechanisms rarely report how much each component contributes, or whether the reported effectiveness holds when the underlying model changes. This article approaches prompt protection as a cross-domain trust problem, composes cryptographic protection, validation, filtering, watermarking, and monitoring across layers and administrative boundaries, and evaluates the resulting architecture across model families and layer configurations.

3 Threat Model and Security Requirements

To make the cross-domain trust perspective operational, this section formalizes the threat model for multi-tenant LLM deployments. The setting separates deployment from ownership across administrative boundaries, and the analysis examines how adversaries with different capabilities interact with the system under this constraint. The following subsections define the system model, characterize the adversary classes, and state the security requirements together with the attack vectors they must withstand.

3.1 System Model

The architecture is defined over a *two-account boundary*. The *Vendor Account* is trusted and controlled by the Software-as-a-Service (SaaS) vendor; it hosts the prompts, the cryptographic keys, the container registry, and the monitoring infrastructure. The *Customer Account* is semi-trusted; it hosts the deployed agent, an Application Programming Interface (API) gateway, and an event-forwarding component. The boundary maps directly onto Amazon Web Services (AWS) account boundaries, Azure subscription or tenant boundaries, and Google Cloud Platform (GCP) project boundaries, so the model does not depend on a particular provider.

3.2 Adversary Classes

Three adversaries with increasing capabilities are considered. The **Customer Administrator** ($\mathcal{A}_1$) holds full privileges over the customer account: it can inspect deployed resources and container images, read environment variables, and intercept traffic. Its motivation ranges from curiosity to competitive intelligence or the replication of the agent without the vendor. The **External Attacker** ($\mathcal{A}_2$) reaches only the public API of the agent and relies on prompt injection, jailbreaking, and social engineering through the model’s input–output interface. The **Malicious Insider** ($\mathcal{A}_3$) is a member of either organization with partial access, for instance to development environments, CI/CD pipelines, or logging systems, who attempts to exfiltrate prompt intellectual property.

3.3 Security Requirements and Attack Mapping

The architecture must provide three guarantees. *Prompt confidentiality* requires that the plaintext prompt never be accessible inside the customer account, neither at rest nor in transit. *Output integrity* requires that responses reflect the vendor’s prompt and that any tampering be detectable. *Forensic traceability* requires that every response carry an invisible per-request fingerprint, so that leaked content can be attributed to an account, a session, and a request. Table 1 maps each attack vector to the adversaries able to execute it, to the guarantee it targets, and to the defense layer that counters it. Figure 1 depicts the same mapping as request flows and contrasts legitimate usage with the attack scenarios. The table uses the abbreviations Hash-based Message Authentication Code (HMAC), man-in-the-middle (MITM), and zero-width character (ZWC).

Table 1: Attack vectors, adversary capabilities, and defense layer mapping.

Attack vector	Adv.	Goal	Defense
Console/API inspection	$\mathcal{A}_1$	Conf.	L1: Cross-acct isolation
Container reverse eng.	$\mathcal{A}_1$	Conf.	L1: Build-time secret
Direct gatekeeper call	$\mathcal{A}_1$	Conf.	L1: HMAC + account binding
Request replay	$\mathcal{A}_1, \mathcal{A}_2$	Conf.	L1: Timestamp + nonce
MITM tampering	$\mathcal{A}_1$	Integ.	L1: Body hash in HMAC
Prompt injection / jailbreak	$\mathcal{A}_2$	Conf.	L2: Risk scoring
Multi-turn extraction	$\mathcal{A}_2$	Conf.	L2+L3: Score + filter
Output exfiltration	$\mathcal{A}_2, \mathcal{A}_3$	Conf.	L3: Canary + detection
Unauthorized redistribution	$\mathcal{A}_3$	Trace.	L3: ZWC fingerprint
Encoding evasion	$\mathcal{A}_2$	Conf.	L2: Encoding detectors
Typoglycemia attacks	$\mathcal{A}_2$	Conf.	L2: Normalization
Monitoring evasion	$\mathcal{A}_1$	All	L4: Vendor-side logs

Trust assumptions. The analysis rests on four assumptions: (1) the vendor’s Identity and Access Management (IAM) configuration is correct; (2) all cross-account communication uses Transport Layer Security (TLS) 1.2 or later; (3) the container runtime isolates process memory; and (4) the LLM API does not retain prompts beyond the session. Assumption (1) is weaker

than the hardware attestation of a TEE, but it holds on every cloud provider without additional hardware.

4 Defense-in-Depth Architecture

The architecture composes four defense layers into a sequential trust chain, in the same way that network security pipelines chain firewalls, Intrusion Detection Systems (IDS), Web Application Firewalls (WAF), and Security Information and Event Management (SIEM) systems. Figure 2 shows the two-account deployment model and separates the trusted vendor-side components from the customer-controlled infrastructure.

The *Vendor Account* contains the trusted components: a secrets store whose prompts are encrypted with a Key Management Service (KMS) key, a serverless gatekeeper in charge of authentication and prompt delivery, a container registry, and a central event bus. The *Customer Account* contains the deployed components: the containerized agent, an API gateway with rate limiting, and an event-forwarding mechanism. The agent obtains its prompt at runtime and keeps it exclusively in process memory. This separation enforces an invariant on which the whole design depends: the system prompt is never persisted in customer-controlled storage and exists only transiently during inference.

Figure 2 describes where components are deployed; the security of the architecture also depends on how requests are processed across them. Preserving prompt confidentiality across accounts requires a sequence of coordinated defenses applied at different stages of the interaction. Figure 3 shows this defense-in-depth pipeline as four sequential layers whose composition covers the complete request lifecycle.

Each layer covers a distinct part of the attack surface. Layer 1 secures prompt delivery through cryptographic request validation. Layer 2 applies OWASP-aligned input validation to detect adversarial inputs before they reach the model. Layer 3 filters model outputs and embeds forensic fingerprints to prevent leakage and support attribution. Layer 4 centralizes cross-account monitoring to enable anomaly detection and coordinated response. Figure 4 details the internal mechanism of each layer; the following subsections describe them in turn, followed by the prompt-level hardening that complements them.

4.1 Layer 1: Cryptographic Prompt Delivery

This layer guarantees that only authenticated agents can retrieve prompts and that requests can be neither tampered with nor replayed. The guarantee follows from a cryptographic binding between the request payload, the originating account, and temporal information. Each request carries an HMAC with SHA-256 (HMAC-SHA256) signature over a structured mes-

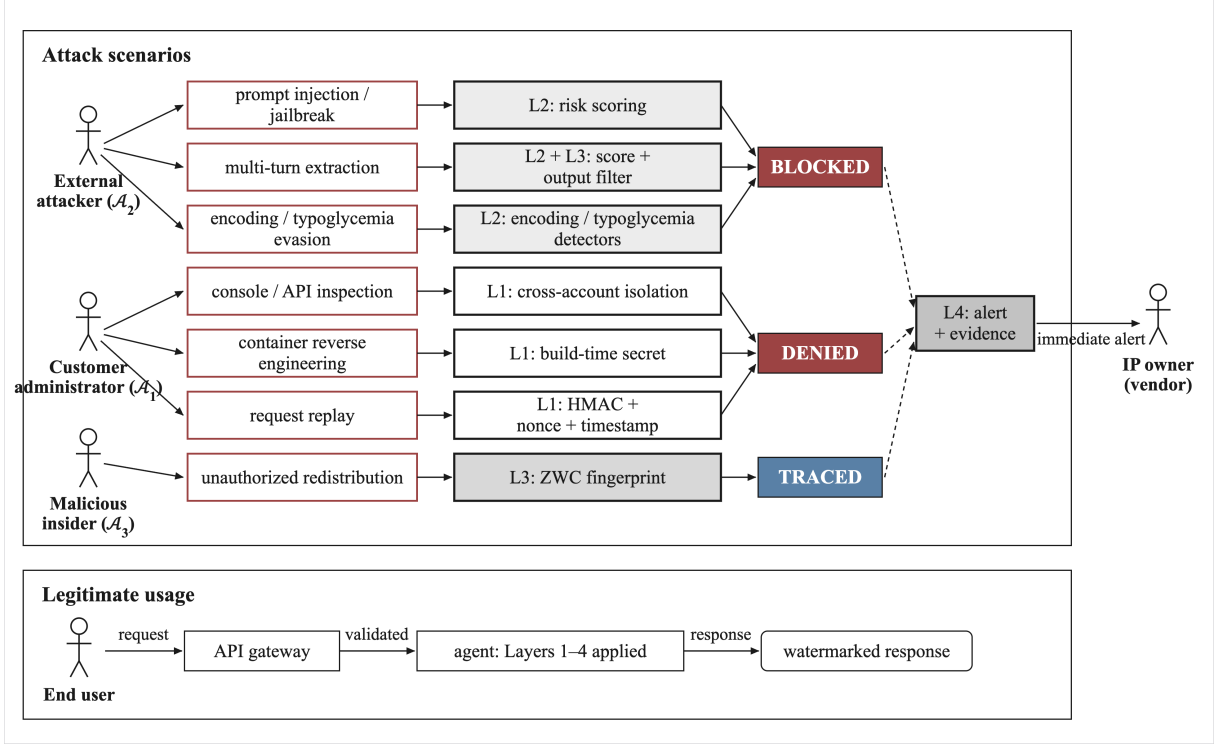


Figure 1: Request flows under legitimate usage and under the attack scenarios of Table 1, with the layer that blocks, denies, or traces each attack and the resulting alert on the vendor side.

sage:

$$m = ts \parallel nonce \parallel account_id \parallel \text{SHA256}(body) \quad (1)$$

The signature is $\sigma = \text{HMAC-SHA256}(k, m)$, where k is a pre-shared secret key. The construction protects the integrity of the request body, binds the request to one account, and provides freshness through the timestamp and the nonce.

Before releasing the prompt, the gatekeeper checks four properties: (1) the timestamp lies within a bounded window, set to five minutes in the reference implementation, which limits replay; (2) the nonce has not been observed before, which a short-lived cache enforces within that window; (3) the account identifier belongs to an allow-list; and (4) the request body matches the signed hash. All comparisons use constant-time operations to avoid timing side channels. The gatekeeper additionally enforces a per-account request rate and raises an alert after repeated failed attempts from the same account. Together with the prompt, it returns an integrity hash (SHA-256 of the prompt content) that the agent verifies on receipt, which extends integrity protection end to end. The signing key is supplied at container build time through a build secret that is never written to an image layer, so it cannot be recovered from the image history.

At the application layer, the mechanism therefore provides request authenticity, payload integrity, and replay protection. These properties keep prompt delivery secure even against adversaries with visibility into

the execution environment.

4.2 Layer 2: OWASP-Aligned Input Validation

This layer mitigates prompt injection and adversarial manipulation by screening user inputs before they reach the model. It relies on a quantitative risk assessment aligned with the OWASP LLM Top 10 [13] and with the complementary taxonomy of the National Institute of Standards and Technology (NIST) [35, 12].

Every incoming message receives a risk score computed from the patterns it matches. The patterns correspond to known attack strategies, such as instruction override, jailbreak personas, prompt extraction requests, and structural injection markers, and are grouped into four severity tiers with additive weights: critical (+30), high (+20), medium (+10), and low (+5). Structural heuristics add smaller increments for unusually long inputs, an excessive number of line breaks, or embedded markup tags. Inputs whose score reaches a fixed threshold, set to 20, are blocked with a fixed refusal message; inputs in the medium band are logged as suspicious and allowed through. The threshold follows from the weights: a score of 20 corresponds to one high-severity pattern or to two independent medium-severity signals, so a single reference to an encoding scheme is tolerated while combinations of weaker signals trigger blocking. This design prevents explicit attacks without over-restricting legitimate technical queries; the empirical support for the

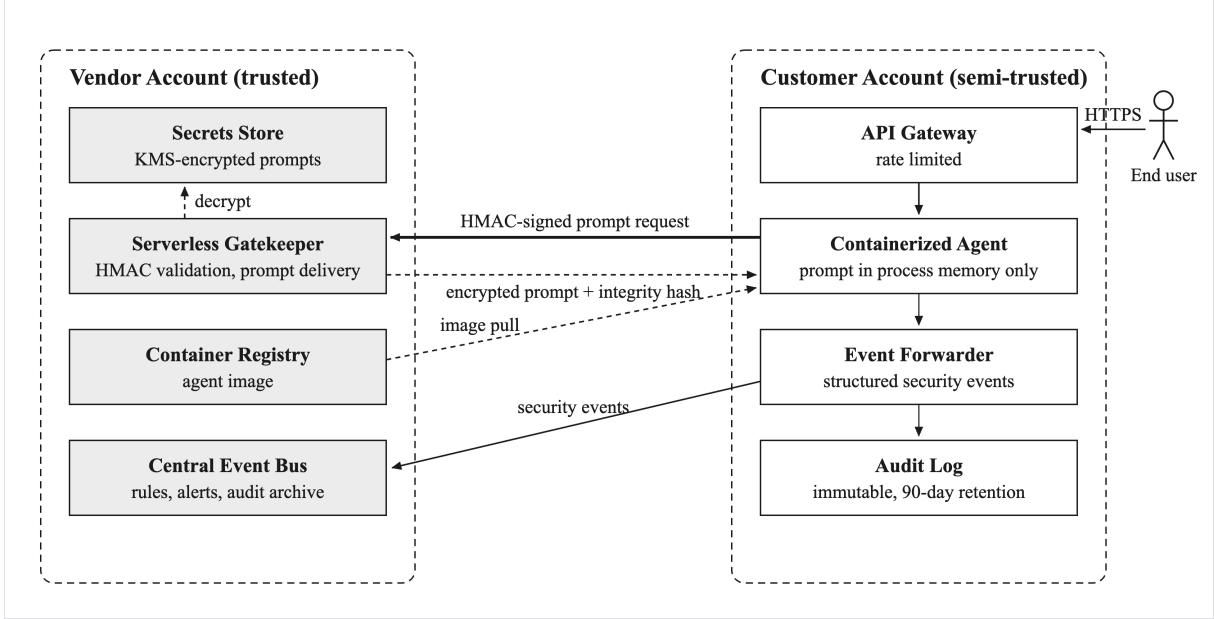


Figure 2: Two-account architecture for cross-domain prompt protection.

operating point is reported in Section 7.2.

Two specialized detectors extend coverage beyond explicit patterns. A typoglycemia detector matches deliberately misspelled variants of security-critical keywords, such as “ignroe” or “systme”, against a dictionary of known variants; two or more variants add +20 to the score. An encoding detector flags payloads hidden through base64, hexadecimal, or Unicode escape sequences, as well as explicit requests to decode ROT13 or similar transformations; any detected encoding adds +20. Before the message is forwarded to the model, a sanitization step replaces known injection prefixes, such as override commands or fake role markers, with a neutral placeholder. Together, these mechanisms form the first line of defense against adversarial inputs: they reduce the likelihood that malicious instructions reach the model while preserving flexibility for legitimate use.

4.3 Layer 3: Output Filtering and Watermarking

This layer prevents the disclosure of sensitive information in model responses and enables forensic attribution of generated content. It processes every response with three complementary mechanisms: leakage detection, active signaling, and covert fingerprinting.

First, responses are inspected for prompt leakage with a set of compiled regular expressions that recognize meta-instructions, role definitions, jailbreak success indicators, and configuration dumps, and with a sliding-window comparison against the delivered prompt: the prompt is divided into 50-character chunks, and a response that reproduces any chunk is treated as a leak. Responses that trigger either check are replaced by a fixed safe message. Second, canary

tokens are embedded in the prompt itself. Each canary is an HTML comment that contains a truncated SHA-256 hash derived from the account identifier and a timestamp; it is placed before and after the prompt body, remains invisible in normal operation, and produces an alert and a blocked response whenever it appears in an output. Third, every response is augmented with a steganographic fingerprint built from zero-width Unicode characters (ZWC) [33, 31]. The fingerprint encodes a hash of the account, session, and request identifiers and is distributed across the generated text, so that it remains imperceptible and resists naive removal. Algorithm 1 shows the embedding procedure of the reference implementation. The same module provides Personally Identifiable Information (PII) detection and redaction routines, which the static test suite exercises.

The mechanisms of this layer provide two complementary guarantees: prevention of prompt leakage at generation time, and post hoc attribution when content is exfiltrated. Proactive filtering and reactive traceability thus reinforce each other.

4.4 Layer 4: Cross-Account Monitoring

This layer provides system-wide observability and coordinated detection of security events across the trust boundary. Monitoring is centralized in the vendor-controlled environment, so adversaries operating inside the customer account cannot suppress or alter security-relevant signals.

Events generated by all layers are forwarded to a central event bus, where they are aggregated and correlated. They include authentication failures, replay attempts, body-hash mismatches, and rate-limit vio-

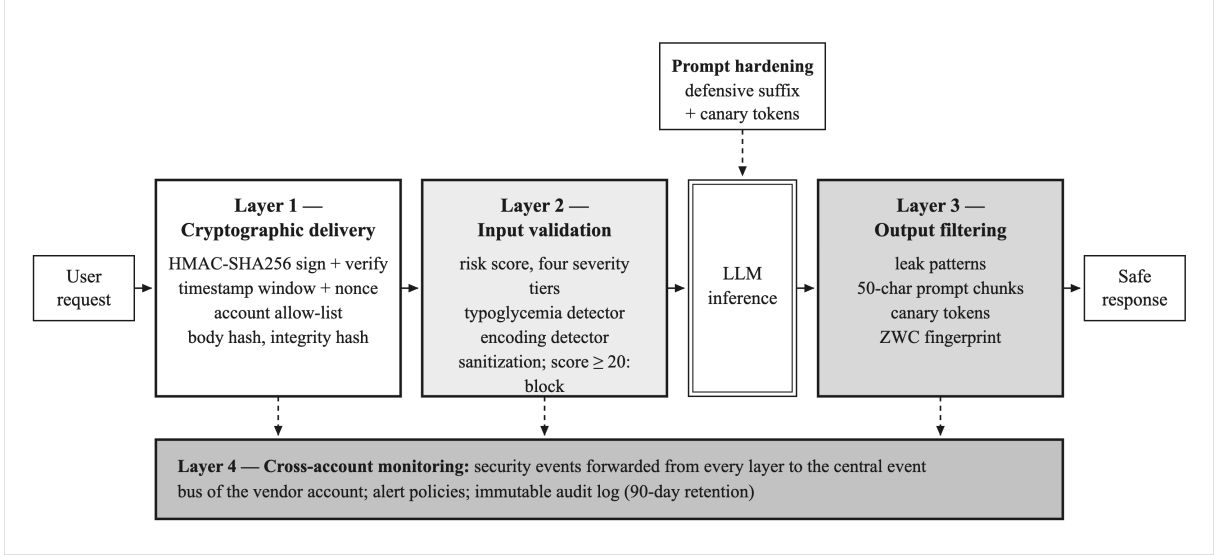


Figure 3: Defense-in-depth pipeline: Layers 1 and 2 act before model inference, Layer 3 acts on the model output, and Layer 4 receives the security events of every layer.

Algorithm 1 ZWC-based watermark embedding

```

1: function WATER-  
   MARK(text, account, session, request)  
2:    $h \leftarrow \text{SHA256}(\text{account} \parallel \text{session} \parallel \text{request})$   
3:    $w \leftarrow \text{EncodeZWC}(h[0:8])$   $\triangleright$  hex digits to  
   ZWC sequences  
4:    $(w_1, w_2, w_3) \leftarrow \text{Split}(w, 3)$   
5:    $\text{words} \leftarrow \text{SplitWords}(\text{text})$   
6:   if  $|\text{words}| > 4$  then  
7:      $P \leftarrow$  three distinct random positions in  
    $[1, |\text{words}|]$   
8:     for  $(p_i, w_i)$  in  $\text{zip}(\text{sorted}(P), (w_1, w_2, w_3))$   
   do  
9:        $\text{words}[p_i] \leftarrow w_i \parallel \text{words}[p_i]$   
10:    end for  
11:    return  $\text{JoinWords}(\text{words})$   
12:  else  
13:    return  $w \parallel \text{text}$   
14:  end if  
15: end function

```

lations from Layer 1; input-validation triggers from Layer 2; and leakage or canary detections from Layer 3. The customer account additionally forwards control-plane events that indicate tampering by a privileged principal, such as modifications of the agent function, changes to the IAM roles that protect it, direct access attempts to the secrets store, alterations of the audit trail, or activation of model-invocation logging. Alerting policies escalate suspicious activity: in the reference implementation, three gatekeeper authentication failures within five minutes, five rate-limit rejections within five minutes, or any replay attempt raise an alert, and every input-validation block in the customer account is forwarded within one minute. All events are

also written to an immutable audit log with a 90-day retention period, so that incidents can be reconstructed even after a partial compromise.

This final layer complements the preceding ones with detection, accountability, and coordinated response across administrative domains.

4.5 Prompt-Level Hardening

Besides the four layers, the prompt delivered by the gatekeeper is hardened in memory before the agent is instantiated. In addition to the canary tokens of Layer 3, a defensive suffix appends eight explicit rules that instruct the model never to reveal, paraphrase, or encode its instructions, to decline extraction attempts framed as roleplay, hypotheticals, or translation, and to answer such attempts with a fixed refusal sentence. This control is inexpensive and requires no infrastructure, but its effect depends entirely on the instruction-following behavior of the underlying model, which the architecture does not control. For that reason it is not counted as a defense layer: it is treated as an explicit factor in the multi-model evaluation and in the ablation study, and the reproducibility environment provides alternative variants of the suffix designed for small open-weight models (Section 5.3).

5 Reference Implementation and Reproducibility

The implementation is a central part of the contribution: it shows that the architecture can be built from standard managed services, and it provides the artifacts used in every experiment reported in this article. The reference implementation and the experimental

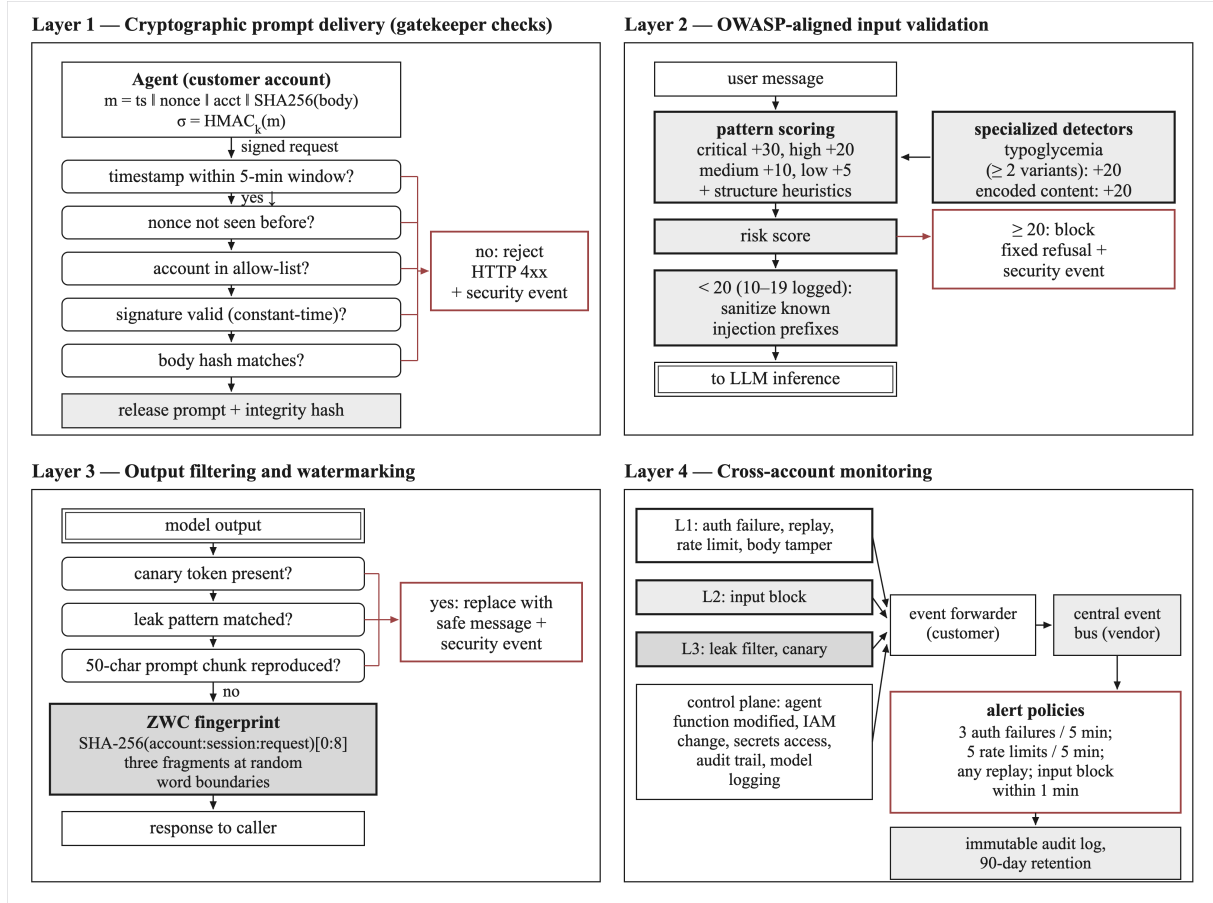


Figure 4: Internal mechanism of each defense layer as implemented in the reference implementation: validation order of the gatekeeper (Layer 1), scoring and blocking decision (Layer 2), output checks and fingerprint embedding (Layer 3), and event sources, forwarding, alerting, and audit (Layer 4).

artifacts are publicly available¹.

5.1 Cloud Deployment

The system is built on serverless services, which provide automatic scaling and cost-efficient operation, and is defined entirely as Infrastructure-as-Code (IaC) with Terraform. Two environments correspond to the two accounts of the model. The central environment provisions the vendor-side components: the encrypted secrets store and its KMS key, the gatekeeper function behind an HTTP endpoint, the container registry, the central event bus with its rules, the alerting topics, and the audit storage. The client environment provisions the customer-side components: the agent function packaged as a container image, the API gateway with throttling, the event forwarder, and the detection rules described in Section 4.4. The architecture relies only on service primitives that all major providers offer; Table 2 lists the AWS services of the reference implementation and their equivalents on Azure and Google Cloud.

¹<https://github.com/dpetrocelli/defense-in-depth-agent-ip>

The agent is a Python application that exposes an HTTP interface with three endpoints, for health checks, tool listing, and invocation, and runs both as a serverless function and as a local process. It is built on an agent framework with tool calling and ships with four demonstration tools (arithmetic, current time, string utilities, and Fibonacci numbers), which makes the injection surface realistic. In the cloud deployment the model is Amazon Nova Lite, invoked through the provider’s managed inference service. Layer 1 is split between the gatekeeper function in the vendor account and a signing client inside the agent; Layers 2 and 3 are implemented as a self-contained filtering module inside the agent; Layer 4 combines the event forwarder, the event-bus rules, and the alarms of both accounts.

Deployment proceeds in three phases. In the *build* phase, the container image is produced with the signing key mounted as a build secret, so that the key is available while the image is built but absent from every image layer. In the *provisioning* phase, the customer-side resources are instantiated through cross-account access policies that allow the customer function to pull the image from the vendor registry without granting that permission to the customer administrator. In the

Table 2: Architectural components, services used in the reference implementation, and equivalent services on other providers.

Component	AWS (reference)	Azure	Google Cloud
Secrets store + KMS	Secrets Manager + KMS	Key Vault + Managed HSM	Secret Manager + Cloud KMS
Serverless gatekeeper	Lambda + API Gateway	Functions + API Management	Cloud Functions (2nd gen)
Container registry	ECR	Container Registry	Artifact Registry
Cross-account event bus	EventBridge	Event Grid	Eventarc
Containerized agent (FaaS)	Lambda (container image)	Functions (container)	Cloud Run (container)
Immutable audit log	S3 Object Lock + CloudWatch Logs	Immutable Blob Storage + Monitor	Cloud Logging + retention

activation phase, the agent retrieves its prompt from the gatekeeper over the authenticated channel, verifies the integrity hash, applies the hardening of Section 4.5, and keeps the result only in process memory. Before serving traffic, a fail-closed pre-flight check verifies that model-invocation logging is disabled in the customer account, which enforces trust assumption (4) of Section 3.

5.2 Security Configuration

Every defense mechanism is governed by a feature flag exposed as an environment variable of the agent (Table 3). All flags default to enabled, and disabling one of them changes only the corresponding mechanism, which allows layer configurations to be compared without code changes. The input sanitization step and canary detection are not governed by flags and remain active in every configuration. This configuration surface is the basis of the ablation study of Section 7.4.

Table 3: Feature flags of the reference implementation and the mechanism each one governs.

Flag	Mechanism governed
USE_GATEKEEPER	L1: prompt retrieval through the gatekeeper
ENABLE_INPUT_VALIDATION	L2: risk scoring and specialized detectors
BLOCK_HIGH_RISK_INPUTS	L2: blocking of high and critical inputs
ENABLE_RESPONSE_FILTER	L3: leakage detection on model outputs
ENABLE_WATERMARKING	L3: ZWC fingerprint embedding
ENABLE_PREFLIGHT_CHECK	Pre-flight model-logging check

5.3 Local Reproducibility Environment

To allow experimentation without cloud accounts, the repository provides a Docker Compose environment that combines LocalStack for cloud-service emulation and Ollama for local LLM inference. A single-account variant runs the agent against an emulated secrets store with the gatekeeper disabled. A multi-account variant reproduces the two-account model with two isolated LocalStack instances: a central instance hosts the gatekeeper function behind an API endpoint, the secrets store, the KMS key, and an alerting topic with a queue that allows Layer 4 alerts to be verified; a client instance provides a separate identity for the agent. The gatekeeper executes the same code as in the cloud deployment, so all four layers are exercised end to end

without code modifications.

The model is selected through a configuration variable, and any model served by Ollama can be used. The agent uses the native Ollama driver of the agent framework, which transmits the system prompt and the tool definitions as separate fields of the request; an earlier configuration based on a generic proxy library silently dropped the system message when tools were present, which nullified the prompt-level hardening. The environment also provides three alternative defensive suffixes, a minimal variant, a variant that states an explicit boundary between protected and normal requests, and an example-based variant, designed for small open-weight models that either ignore the production suffix or over-apply it. Mechanisms that depend on model behavior, in particular the defensive suffix and the model’s own refusals, remain sensitive to the underlying LLM, which is precisely what the multi-model evaluation measures. The repository also contains the experimental tooling: the static penetration-test suite, the attack and legitimate-query corpus with its runner, the false-positive corpus, and the latency benchmark.

6 Experimental Methodology

6.1 Evaluation Dimensions and Metrics

Security effectiveness is measured on two complementary levels. Controlled penetration testing verifies each mechanism against a fixed set of test cases with a pass/fail criterion. Live evaluation submits attack and legitimate queries to a deployed agent and measures the *block rate*, the share of attack queries that do not obtain a substantive answer, and the *false-positive rate* (FPR), the share of legitimate queries that are blocked. The new experiments add the *attack success rate* (ASR), the share of attack queries whose final response leaks protected content, so that blocking and leakage are measured separately. Performance overhead is measured as the latency added by each layer at the 50th, 95th, and 99th percentiles (P50, P95, P99) and as the throughput under concurrent load. Deployability is assessed through the monthly infrastructure cost of the defense layers on three providers.

6.2 Penetration Test Suite

The controlled suite comprises 63 test cases in 12 categories aligned with the OWASP LLM Top 10 [13] and the NIST taxonomy of adversarial machine learning [35]. Every test specifies the attack vector, the targeted security goal, the defense layer expected to intercept it, the payload, and the pass/fail criterion. The categories cover representative attack patterns from established taxonomies rather than an exhaustive adversarial exploration.

6.3 Attack Corpus and Legitimate Queries

The live evaluations use a corpus of 42 attack queries organized in eight categories of increasing sophistication: naive extraction (8), instruction override (5), jailbreak and persona change (7), encoded payloads (4), typoglycemia (4), social engineering (5), indirect injection through fake system markers (4), and multi-turn escalation (5). The corpus is complemented by 23 legitimate queries in three length groups (10 short, 10 medium, and 3 long), which are used to measure false positives. In the live evaluation of Section 7.2, an attack query counts as blocked when the request is rejected, when the response is empty, or when the response consists of a refusal; a legitimate query counts as a false positive under the same criterion. The multi-model evaluation and the ablation study refine this criterion with the per-request outcome classification of Section 6.5.

6.4 Deployment Environments

Two environments are used. The cloud environment is the two-account AWS deployment of Section 5, with Amazon Nova Lite as the underlying model; it supports the live evaluation, the performance measurements, and the cost analysis. The local environment is the multi-account Docker Compose stack of Section 5.3, which runs the same agent and gatekeeper code against open-weight models served by Ollama; it supports the multi-model evaluation and the ablation study. In both environments the experiments use the test prompts of the corpus rather than a production prompt.

6.5 Multi-Model Evaluation Protocol

The multi-model evaluation answers RQ1 by holding every element of the pipeline fixed and varying only the underlying LLM, so that differences in outcome can be attributed to the model. Table 4 lists the nine models, which span six families and both deployment modes of the architecture: Amazon Nova Lite is invoked through the managed inference service of the cloud environment and serves as the reference model of the live evaluation, while the eight open-weight models are served by Ollama in the local environment. All models run the same agent and gatekeeper code.

The following factors are fixed across models. The base system prompt is the medium-length prompt of the corpus (about 600 characters), whose closing sentence is a minimal confidentiality instruction that forms part of the base prompt in every configuration. The prompt is hardened with the production defensive suffix and the canary tokens of Section 4.5, and the agent exposes its four demonstration tools. Decoding is deterministic where the runtime allows it (temperature 0, nucleus sampling disabled), with identical parameters for all models. All four layers are enabled, which corresponds to configuration C4 of Table 5. Every query is submitted in a fresh conversation, with the exception of the multi-turn escalation category, whose five queries form one session and are submitted in their listed order. Each model is evaluated in three independent runs; results are reported as mean and standard deviation. Requests that end in a transport error or a timeout are counted as errors, reported separately, and excluded from the denominators.

The outcome of every request is classified by the mechanism that determined it: *L2 block* (risk score at or above the threshold), *model refusal* (the model declines before any filtering), *L3 filter* (the response filter replaced the output), *canary* (a canary token appeared in the output), *pass* (a substantive response reached the caller), or *error*. Because Layer 2 blocks, canary detections, and model refusals return the same refusal sentence, the label is recorded by the agent as a structured audit field rather than inferred from the response text; the raw model output produced before Layer 3 is recorded as well. Leakage is assessed offline with the same criterion that Layer 3 applies at runtime: a response leaks if it contains a canary token or reproduces any 50-character chunk of the base prompt, ignoring case. The oracle is applied both to the final response, which yields the ASR, and to the raw model output, which yields the *pre-filter leakage* of the model. The block rate is the share of attack queries whose outcome is an L2 block, a model refusal, an L3 filter, or a canary detection; the FPR is the share of legitimate queries with any of these outcomes. Results are reported as overall rates per model (Table 8), as block rates per attack category (Table 9), and as the attribution of outcomes to mechanisms (Table 10).

6.6 Defense-Layer Ablation Protocol

The ablation study answers RQ2 by measuring how the security outcome changes when individual defenses are removed from the pipeline. The layers are not interchangeable classifiers. Layer 1 governs authentication, request integrity, account binding, and replay protection, and Layer 4 governs monitoring, auditability, and coordinated response; neither acts on the content of a request or a response, so removing them does not change the outcome of the attack corpus. Both are therefore kept active in every configuration and verified separately, with attack cases of their own, in

Table 4: Models evaluated under the common protocol. Parameter counts are those reported by the providers; quantization and inference hardware are reported per run.

Family	Model	Identifier	Params	Runtime	Quantization / hardware
Amazon Nova	Nova Lite	amazon.nova-lite-v1:0	undisclosed	Amazon Bedrock (cloud)	managed / TBD
Llama	Llama 3.2 1B	llama3.2:1b	1B	Ollama (local)	TBD
Llama	Llama 3 8B	llama3:8b	8B	Ollama (local)	TBD
Gemma	Gemma 3 4B	gemma3:4b	4B	Ollama (local)	TBD
Gemma	Gemma 4 E4B	gemma4:e4b	4B (effective)	Ollama (local)	TBD
Phi	Phi-4 Mini	phi4-mini	3.8B	Ollama (local)	TBD
Mistral	Mistral 7B	mistral:7b	7B	Ollama (local)	TBD
Qwen	Qwen 3.5 4B	qwen3.5:4b	4B	Ollama (local)	TBD
Qwen	Qwen 3.5 9B	qwen3.5:9b	9.6B	Ollama (local)	TBD

Table 14. The mechanisms that act on the input–output path of a request, and hence on the attack corpus, are the defensive suffix, Layer 2, and Layer 3; the ablation varies these.

Table 5 defines the configurations. C0 disables every content defense and measures the floor set by the model itself, since the base prompt retains only its closing confidentiality sentence. C1 adds the defensive suffix alone and isolates the model-side control. C2 and C3 add Layer 2 and Layer 3, respectively, on top of the defensive suffix. C4 is the deployed pipeline and coincides with the condition of the multi-model evaluation. C4[†] keeps the pipeline of C4 but runs Layer 2 in detect-only mode, in which the risk score is computed and recorded while blocking is disabled; every attack therefore reaches the model and Layer 3, and the verdicts of both layers are observed on the same request. The configurations map directly onto the feature flags of Table 3; the two mechanisms that the reference implementation applies unconditionally, input sanitization and canary detection, follow the Layer 2 and Layer 3 settings, respectively. C0 and C1 expose only the test prompt of the corpus and are executed in the local environment.

All configurations are evaluated for the nine models of Table 4, with the fixed factors, repetitions, outcome classification, and metrics of Section 6.5. The individual contribution of a mechanism is the change in block rate and ASR between configurations that differ only in that mechanism: the defensive suffix from C0 to C1, Layer 2 from C1 to C2 and from C3 to C4, and Layer 3 from C1 to C3 and from C2 to C4. Complementarity is measured on C4[†]: each attack query is labeled with the verdict of Layer 2 (would block or pass) and with its downstream outcome (intercepted by Layer 3 or by a canary, refused by the model, or passed), which yields the contingency table of Table 12. Queries that pass Layer 2 and are intercepted by Layer 3 are the direct evidence of complementarity; queries that pass both are evaluated with the leakage oracle to quantify residual exposure. Results are reported per configuration and model in Table 11 and per attack category for the cloud reference model in Table 13.

Layer 1 is verified with deterministic cases executed against the gatekeeper of the multi-account environ-

ment, which runs the production code: a valid signed request must return the prompt with a valid integrity hash, and each violated property must produce the response that follows from the validation order of the gatekeeper without disclosing prompt content. Layer 4 is verified by triggering each event class and checking that the event reaches the central bus and, where an alerting policy applies, that an alert is published on the alert topic, which the environment exposes through a subscribed queue.

7 Security Evaluation

7.1 Controlled Penetration Testing

Table 6 reports the outcome of the 63 test cases described in Section 6, grouped by category, together with the OWASP item they exercise and the layer expected to intercept them.

All authentication bypass attempts against Layer 1, namely expired timestamps, duplicate nonces, forged signatures, and missing headers, were rejected with Hypertext Transfer Protocol (HTTP) 403 responses that disclosed no information. Layer 2 handled the 44 input-facing tests, which range from naive extraction requests to multi-turn social engineering and encoding evasion; in particular, the typoglycemia detector blocked obfuscated variants that defeat a naive regular-expression approach. Layer 3 provided complementary protection: the sliding-window comparison detected partial leakage of fragments of 50 or more characters that the pattern-based check alone did not capture. Finally, eight tests targeted boundary conditions between layers and confirmed that attacks crafted to lower their Layer 2 score were still mitigated by the Layer 3 mechanisms. The jailbreak category includes the “Do Anything Now” (DAN) persona attacks.

7.2 Live Two-Account Deployment

To validate the architecture under realistic conditions, the full four-layer stack was deployed across two separate cloud accounts, vendor and customer, and exposed to the 42 attack queries and the 23 legitimate queries of Section 6.3. Table 7 reports the outcome per category.

Table 5: Ablation configurations. Layer 1 (gatekeeper delivery) and Layer 4 (monitoring) remain active in every configuration, since they do not act on the input–output path of a request; they are verified separately (Table 14). DP denotes the defensive suffix of Section 4.5.

Config.	Description	DP	L2 scoring	L2 blocking	Sanitization	L3 filter	Canary	Watermark
C0	No defenses (model-only floor)	–	–	–	–	–	–	–
C1	DP only	✓	–	–	–	–	–	–
C2	DP + Layer 2	✓	✓	✓	✓	–	–	–
C3	DP + Layer 3	✓	–	–	–	✓	✓	✓
C4	DP + Layer 2 + Layer 3 (deployed pipeline)	✓	✓	✓	✓	✓	✓	✓
C4 [†]	As C4 with Layer 2 in detect-only mode	✓	✓	–	✓	✓	✓	✓

Table 6: Penetration test results (63 tests, 100% pass rate).

Category	OWASP	Tests	Pass	Layer
Direct extraction	LLM01/06	8	8	L2+L3
Multi-turn extraction	LLM01	5	5	L2+L3
Jailbreak / DAN	LLM01	7	7	L2
Instruction override	LLM01	6	6	L2
Prompt injection	LLM01	6	6	L2
Encoding evasion	LLM01	4	4	L2
Typoglycemia evasion	LLM01	3	3	L2
Social engineering	LLM01	5	5	L2
Output leakage	LLM02/06	6	6	L3
PII exfiltration	LLM06	4	4	L3
Response safety	LLM02	5	5	L3
Replay / auth bypass	—	4	4	L1
Total		63	63	

Table 7: Live deployment: attack detection by category (42 attacks, 23 legitimate queries). FP = false positive.

Attack category	Blocked	Rate	Defense
Override commands	5/5	100%	L2 (risk score)
Jailbreak / DAN	7/7	100%	L2 (risk score)
Encoded payloads	4/4	100%	L2 (encoding det.)
Typoglycemia	4/4	100%	L2 (typo det.)
Social engineering	5/5	100%	L2+L3
Indirect injection	4/4	100%	L2 (structural)
Naive extraction	6/8	75%	L2+L3+defensive prompt
Multi-turn	3/5	60%	L2+L3 (escalation)
Total attacks	38/42	90.5%	
Legitimate (FP)	0/23	0.0%	

The four queries that were not blocked are reconnaissance probes that obtain no prompt content; the architecture tolerates ambiguous interactions while it prevents the disclosure of protected information. Multi-turn escalation has the lowest rate because input validation is stateless: every request is scored on its own, and escalation sessions spread their intent over turns that are individually benign, so that only one of the five sessions contains a turn that reaches the blocking threshold by itself. The sessions that were blocked were intercepted by the combination of the escalation heuristics of Layer 2 and the output-side leakage filter of Layer 3 once the responses started to approach protected content. Cumulative session-level risk scoring across turns is a natural extension that would close this gap.

Threshold sensitivity. A sensitivity analysis over the evaluation corpus supports the operating point of Layer 2. The coverage of the score-based detector is

stable between thresholds 15 and 20 (20 of 42 attacks) and falls sharply at 25 (12 of 42), while no legitimate query scores above 5, which leaves a four-fold margin below the blocking threshold; the attack categories not covered by the generic scorer are handled by the specialized detectors. The detector set was also calibrated iteratively against a preliminary live run, which raised the overall blocking rate from 59.5% to the final 90.5% while keeping false positives at zero.

Concurrency. Under concurrent load, throughput remained stable up to 50 concurrent requests (13.8 requests per second); at higher concurrency it degraded because of infrastructure limits of the serverless configuration.

7.3 Multi-Model Evaluation

This section reports the evaluation of the full pipeline across the nine models of Table 4 under the protocol of Section 6.5. The comparison addresses RQ1: since the corpus, the prompt, the hardening, the decoding parameters, and the four layers are identical for every model, the variation observed across rows is attributable to the underlying LLM. Table 8 summarizes block rate, ASR, and FPR per model; Table 9 decomposes the block rate by attack category, which shows whether the categories that depend on model behavior, such as naive extraction and multi-turn escalation, behave differently from the categories that the deterministic detectors of Layer 2 cover; and Table 10 attributes every outcome to the mechanism that produced it, which separates the model-independent contribution of Layer 2 from the contributions of the model’s own refusals and of Layer 3.

[TODO: Add the interpretation of Tables 8–10 after the experimental results are available: differences across model families, model-dependent versus model-independent categories, false-positive behavior, and pre-filter leakage.]

7.4 Defense-Layer Ablation Study

This section reports the ablation study defined in Section 6.6, which addresses RQ2. Table 11 gives block rate, ASR, and FPR for every configuration of Table 5 and every model, so that the contribution of each mechanism is read as the difference between adjacent

Table 8: Multi-model evaluation with the full pipeline (configuration C4): blocked attacks, block rate, attack success rate (ASR, leaked final responses), false positives (FP) on the 23 legitimate queries, and false-positive rate (FPR). Mean $\pm$ standard deviation over three runs; requests that ended in transport errors or timeouts are excluded from the denominators.

Model	Blocked (n/42)	Block rate (%)	ASR (%)	FP (n/23)	FPR (%)
Nova Lite	TBD	TBD	TBD	TBD	TBD
Llama 3.2 1B	TBD	TBD	TBD	TBD	TBD
Llama 3 8B	TBD	TBD	TBD	TBD	TBD
Gemma 3 4B	TBD	TBD	TBD	TBD	TBD
Gemma 4 E4B	TBD	TBD	TBD	TBD	TBD
Phi-4 Mini	TBD	TBD	TBD	TBD	TBD
Mistral 7B	TBD	TBD	TBD	TBD	TBD
Qwen 3.5 4B	TBD	TBD	TBD	TBD	TBD
Qwen 3.5 9B	TBD	TBD	TBD	TBD	TBD

configurations: the defensive suffix between C0 and C1, Layer 2 between C1 and C2 or between C3 and C4, and Layer 3 between C1 and C3 or between C2 and C4. Table 12 reports, for the detect-only configuration C4[†], how the verdict of Layer 2 and the downstream outcome combine on each attack query; its central cells are the queries that pass Layer 2 and are intercepted by Layer 3, which constitute the direct evidence for the complementarity on which the defense-in-depth claim rests, and the queries that pass both layers, whose leakage quantifies the residual exposure. Table 13 decomposes the ablation by attack category for the cloud reference model, and Table 14 reports the verification of Layers 1 and 4 with their own attack cases.

[TODO: Add the interpretation of Tables 11–14 after the experimental results are available: individual contribution of the defensive suffix, Layer 2, and Layer 3; complementarity between Layer 2 and Layer 3; residual exposure; categories that depend on each mechanism; and the outcome of the Layer 1 and Layer 4 verification.]

8 Performance and Cost Evaluation

8.1 Latency Overhead

Table 15 reports the latency added by each defense layer in the AWS reference implementation over $n = 1,000$ requests.

The total overhead at P99 is 68.3 ms, well under the 100–200 ms range that is imperceptible in conversational applications. Cold starts of the container-based function average 1.2 s; provisioned concurrency removes them at approximately \$4.50 per month and per instance, a cost that is not part of the base infrastructure cost reported next.

8.2 Infrastructure Cost

Table 16 reports the monthly infrastructure cost of the defense layers on three cloud providers at 50,000

requests per month. Function-as-a-Service (FaaS) denotes the serverless execution of the gatekeeper component. The figures are *independent of model inference*: the defenses operate at the transport and application layers and do not modify inference calls, so the cost of LLM invocation is unchanged.

The defense layers add a minimal infrastructure cost that is negligible relative to model inference. The architecture is therefore economically feasible for real deployments, including those of moderate scale.

9 Discussion and Limitations

[TODO: Add a paragraph on the implications of the multi-model and ablation results for the defense-in-depth claim after the experimental results are available.]

From a systems perspective, the proposed trust model exchanges hardware-backed guarantees for broader deployability. Its root of trust is a correctly configured cloud IAM, which is weaker than hardware attestation but available on every provider. The defense-in-depth design mitigates this weakness: when one layer is compromised, for example through an IAM misconfiguration, the remaining layers continue to enforce protection. In this sense the architecture and TEE-based approaches are complementary rather than competing. Recent measurements show that TEE overhead has fallen considerably, below 10% on CPUs and below 7% on GPUs [20], so high-assurance deployments may combine both approaches.

The architecture generalizes across cloud providers, since every component maps to an equivalent service on Azure and GCP (Table 2), and portability does not alter the design principles. Several limitations remain. Rotating the HMAC signing key currently requires rebuilding the container image. Risk-scoring thresholds may need tuning per deployment. Text-processing pipelines that normalize Unicode may strip the zero-width watermark. The architecture has not been evaluated against adversarial fine-tuning, in which a model is retrained on captured prompt–response pairs, and cold-start latency in serverless environments may require provisioned concurrency. The adversarial evaluation is also static: runtime memory scraping by a privileged customer administrator ($\mathcal{A}_1$), who could read the decrypted prompt from process memory, was not tested, and neither were adaptive adversaries that refine their inputs from the observed blocking behavior. Per-account watermarked prompt variants make such leaks attributable rather than preventable, and adaptive red-teaming remains future work. Findings on steganographic collusion [34] further indicate that covert signaling mechanisms carry risks of their own, which motivates additional monitoring.

From an ethical standpoint, the watermarking mechanism identifies accounts and sessions rather than individual users. Its use should be disclosed transparently

Table 9: Block rate per attack category and model with the full pipeline (configuration C4), as blocked/total over the three runs.

Attack category	Nova Lite	Llama 3.2 1B	Llama 3 8B	Gemma 3 4B	Gemma 4 E4B	Phi-4 Mini	Mistral 7B	Qwen 3.5 4B	Qwen 3.5 9B
Naive extraction (8)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Override commands (5)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Jailbreak / DAN (7)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Encoded payloads (4)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Typoglycemia (4)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Social engineering (5)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Indirect injection (4)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Multi-turn escalation (5)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Total (42)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD

Table 10: Attribution of the outcome of the 42 attack queries per model (configuration C4): mechanism that determined the outcome of each request, and leakage among the requests that were not intercepted. Counts summed over the three runs.

Model	L2 block	Model refusal	L3 filter	Canary	Passed (no leak)	Passed (leak)
Nova Lite	TBD	TBD	TBD	TBD	TBD	TBD
Llama 3.2 1B	TBD	TBD	TBD	TBD	TBD	TBD
Llama 3 8B	TBD	TBD	TBD	TBD	TBD	TBD
Gemma 3 4B	TBD	TBD	TBD	TBD	TBD	TBD
Gemma 4 E4B	TBD	TBD	TBD	TBD	TBD	TBD
Phi-4 Mini	TBD	TBD	TBD	TBD	TBD	TBD
Mistral 7B	TBD	TBD	TBD	TBD	TBD	TBD
Qwen 3.5 4B	TBD	TBD	TBD	TBD	TBD	TBD
Qwen 3.5 9B	TBD	TBD	TBD	TBD	TBD	TBD

trust chain, and evaluation against adaptive adversaries. Further directions include redundant watermarking that combines ZWC steganography with token-based schemes [30, 32], and stateful detection mechanisms for multi-turn interactions.

and confined to intellectual property protection and forensic analysis, in line with standard privacy practices.

10 Conclusions and Future Work

This article addressed the protection of AI agent intellectual property in multi-tenant cloud deployments as a cross-domain trust problem and presented a defense-in-depth architecture for it. The composition of cryptographic prompt delivery, OWASP-aligned input validation, output filtering with steganographic watermarking, and cross-account monitoring into a sequential trust chain provides prompt confidentiality, output integrity, and forensic traceability with standard cloud services and without TEEs or specialized hardware. The architecture is cloud-agnostic; its serverless reference implementation, deployed across two separate accounts, showed strong security effectiveness against realistic attack scenarios, a low performance overhead, and a negligible infrastructure cost.

[TODO: Add the conclusions of the multi-model evaluation (RQ1) and of the defense-layer ablation study (RQ2) after the experimental results are available.]

Future work will extend the architecture in several directions: optional integration with TEE-based inference for high-assurance deployments, automated rotation of the HMAC signing key without downtime, formal verification of the security properties of the

Table 11: Ablation results per configuration and model: block rate (BR, %), attack success rate (ASR, %), and false-positive rate (FPR, %). Mean over three runs; standard deviations are reported in the repository.

Config.	Metric	Nova Lite	Llama 3.2 1B	Llama 3 8B	Gemma 3 4B	Gemma 4 E4B	Phi-4 Mini	Mistral 7B	Qwen 3.5 4B	Qwen 3.5 9B	Mean
C0	BR	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
	ASR	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
	FPR	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
C1	BR	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
	ASR	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
	FPR	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
C2	BR	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
	ASR	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
	FPR	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
C3	BR	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
	ASR	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
	FPR	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
C4	BR	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
	ASR	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
	FPR	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD

Table 12: Complementarity between Layer 2 and Layer 3 (configuration C4[†], Layer 2 in detect-only mode): number of attack queries by Layer 2 verdict and downstream outcome, summed over the three runs. “L3 intercepts” includes canary detections; “passes” reports the number of leaked responses in parentheses.

Model	Layer 2 would block			Layer 2 passes		
	L3 intercepts	Model refuses	Passes (leak)	L3 intercepts	Model refuses	Passes (leak)
Nova Lite	TBD	TBD	TBD	TBD	TBD	TBD
Llama 3.2 1B	TBD	TBD	TBD	TBD	TBD	TBD
Llama 3 8B	TBD	TBD	TBD	TBD	TBD	TBD
Gemma 3 4B	TBD	TBD	TBD	TBD	TBD	TBD
Gemma 4 E4B	TBD	TBD	TBD	TBD	TBD	TBD
Phi-4 Mini	TBD	TBD	TBD	TBD	TBD	TBD
Mistral 7B	TBD	TBD	TBD	TBD	TBD	TBD
Qwen 3.5 4B	TBD	TBD	TBD	TBD	TBD	TBD
Qwen 3.5 9B	TBD	TBD	TBD	TBD	TBD	TBD

Table 13: Blocked attacks per category and configuration for the cloud reference model (Amazon Nova Lite), as blocked/total over the three runs. The corresponding tables for the other models are provided in the repository.

Attack category	C0	C1	C2	C3	C4
Naive extraction (8)	TBD	TBD	TBD	TBD	TBD
Override commands (5)	TBD	TBD	TBD	TBD	TBD
Jailbreak / DAN (7)	TBD	TBD	TBD	TBD	TBD
Encoded payloads (4)	TBD	TBD	TBD	TBD	TBD
Typoglycemia (4)	TBD	TBD	TBD	TBD	TBD
Social engineering (5)	TBD	TBD	TBD	TBD	TBD
Indirect injection (4)	TBD	TBD	TBD	TBD	TBD
Multi-turn escalation (5)	TBD	TBD	TBD	TBD	TBD
Total (42)	TBD	TBD	TBD	TBD	TBD

Table 14: Verification of Layer 1 and Layer 4 in the multi-account environment. Layer 1 cases are executed against the gatekeeper with the production code; the expected responses follow from its validation order. Layer 4 cases check that each event class reaches the central event bus and, where an alerting policy applies, raises an alert on the alert topic.

Layer	Case	Adversary	Expected outcome	Observed
L1	Valid signed request	–	HTTP 200, prompt with valid integrity hash	TBD
L1	Missing authentication headers	$\mathcal{A}_1, \mathcal{A}_2$	HTTP 400, no prompt content	TBD
L1	Account not in allow-list	$\mathcal{A}_1$	HTTP 403, no prompt content	TBD
L1	Timestamp outside the 5-minute window	$\mathcal{A}_1, \mathcal{A}_2$	HTTP 403, no prompt content	TBD
L1	Replayed nonce	$\mathcal{A}_1, \mathcal{A}_2$	HTTP 403, replay event	TBD
L1	Forged signature	$\mathcal{A}_1$	HTTP 403, no prompt content	TBD
L1	Tampered request body	$\mathcal{A}_1$	HTTP 403, body-tamper event	TBD
L1	Per-account rate limit exceeded	$\mathcal{A}_2$	HTTP 429, rate-limit event	TBD
L4	Three authentication failures within 5 min	$\mathcal{A}_1$	Events on central bus; alert raised	TBD
L4	Replay attempt	$\mathcal{A}_1, \mathcal{A}_2$	Event on central bus; immediate alert	TBD
L4	Five rate-limit rejections within 5 min	$\mathcal{A}_2$	Events on central bus; alert raised	TBD
L4	Layer 2 input block	$\mathcal{A}_2$	Event forwarded within 1 min	TBD
L4	Layer 3 leakage filter	$\mathcal{A}_2$	Event on central bus	TBD
L4	Canary token in output	$\mathcal{A}_2$	Event on central bus; alert raised	TBD
L4	Customer-side tampering (agent function modified)	$\mathcal{A}_1$	Control-plane event forwarded	TBD

Table 15: Latency overhead per defense layer (ms, $n = 1,000$).

Layer	P50	P95	P99
L1: HMAC validation	2.1	3.8	5.2
L2: Risk scoring	8.3	14.7	19.1
L3: Output filter + ZWC	12.5	22.3	31.6
L4: Event forwarding	4.1	8.2	12.4
Total	27.0	49.0	68.3

Table 16: Monthly infrastructure cost of the defense layers by provider (50K requests).

Component	Service primitive	AWS	Azure	GCP
Secrets + Encryption	Managed secrets + KMS key	\$1.40	\$1.60	\$1.06
Gatekeeper + API	FaaS function + HTTP endpoint	\$0.05	\$0.00	\$0.00
Event Bus	Cross-account event routing	\$0.00	\$0.00	\$0.00
Audit Storage	Immutable log storage (90-day)	\$0.25	\$0.25	\$0.25
Monitoring	Log ingestion + retention	\$0.03	\$0.10	\$0.05
Container Registry	Image storage (~100 MB)	\$0.01	\$5.01	\$0.01
Total		\$1.74	\$6.96	\$1.37

Authors' contribution

[TODO: Complete with the CRediT roles agreed by the authors (DP, JMF).] All authors reviewed the results and approved the final version of the manuscript.

Competing interests

The authors have declared that no competing interests exist.

Declaration of Generative AI and AI-assisted Technologies in the Research/Writing Process

[TODO: Select the applicable statement required by the journal.]

Data Availability

The reference implementation, the Terraform modules, the local reproducibility environment, the attack and legitimate-query corpus, the experimental tooling, and the raw results of the experiments are available at <https://github.com/dpetrocelli/defense-in-depth-agent-ip>.

References

- [1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan *et al.*, “Language models are few-shot learners,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [2] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora *et al.*, “On the opportunities and risks of foundation models,” 2021, stanford CRFM report. [Online]. Available: <https://arxiv.org/abs/2108.07258>
- [3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2023.
- [4] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” *arXiv preprint arXiv:2302.04761*, 2023.
- [5] L. Wang, C. Ma, X. Chen, Q. Yang, Q. Zhang *et al.*, “A survey on large language model based autonomous agents,” *arXiv preprint arXiv:2308.11432*, 2024.
- [6] MarketsandMarkets, “AI agents market—global forecast to 2030,” MarketsandMarkets Research, Tech. Rep., 2024, report ID: TC-9261. Available at <https://www.marketsandmarkets.com/Market-Reports/ai-agents-market-15761548.html>.
- [7] European Union, “General data protection regulation (gdpr),” European Union, Tech. Rep., 2016, regulation (EU) 2016/679.
- [8] Amazon Web Services, “Shared responsibility model,” Amazon Web Services, Tech. Rep., 2023. [Online]. Available: <https://aws.amazon.com/compliance/shared-responsibility-model/>
- [9] A. Shahriar, M. N. Rahman, S. Ahmed, F. Sadeque, and M. R. Parvez, “A survey on agentic security: Applications, threats and defenses,” *arXiv preprint arXiv:2510.06445*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.06445>
- [10] B. Hui, H. Yang, H. Yuan, N. Z. Gong, and L. Lyu, “PLeak: Prompt leaking attacks against large language model applications,” in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2024.
- [11] D. Agarwal, A. R. Fabbri, B. Risher, P. Laban, S. Joty, and C.-S. Wu, “Prompt leakage effect and mitigation strategies for multi-turn LLM applications,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP), Industry Track*, 2024.
- [12] B. C. Das, M. H. Amini, and Y. Wu, “Prompt injection attacks in large language models and AI agent systems: A comprehensive review,” *Information*, vol. 17, no. 1, p. 54, 2026.
- [13] OWASP Foundation, “OWASP top 10 for LLM applications 2025,” Open Web Application Security Project, Tech. Rep., 2025.
- [14] M. A. van Wyk, M. Bekker, X. L. Richards, and K. J. Nixon, “Protect your prompts: Protocols for IP protection in LLM applications,” *arXiv preprint arXiv:2306.06297*, 2023. [Online]. Available: <https://arxiv.org/abs/2306.06297>
- [15] Z. Zhuang, M.-I. Nicolae, H.-P. Wang, and M. Fritz, “ProxyPrompt: Securing system prompts against prompt extraction attacks,” *arXiv preprint arXiv:2505.11459*, 2025. [Online]. Available: <https://arxiv.org/abs/2505.11459>
- [16] B. Cao, C. Li, Y. Cao, Y. Ge, T. Wang, and J. Chen, “You can’t steal nothing: Mitigating prompt leakages in LLMs via system vectors,” in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’25. Association for Computing Machinery, 2025, pp. 4423–4437. [Online]. Available: <https://doi.org/10.1145/3719027.3765124>
- [17] S.-H. Chan, “Encrypted prompt: Securing LLM applications against unauthorized actions,” *arXiv preprint arXiv:2503.23250*, 2025. [Online]. Available: <https://arxiv.org/abs/2503.23250>
- [18] A. Alzahrani, “PromptGuard: A structured framework for injection-resilient language models,” *Scientific Reports*, 2026. [Online]. Available: <https://www.nature.com/articles/s41598-025-31086-y>
- [19] Z. Jiang, Z. Jin, and G. He, “PromptKeeper: Safeguarding system prompts for LLMs,” in *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, 2025, pp. 2712–2728. [Online]. Available: <https://aclanthology.org/2025.findings-emnlp.147/>
- [20] M. Chrapek, M. Copik, E. Mettaz, and T. Hoefler, “Confidential LLM inference: Performance and cost across CPU and GPU TEEs,” in *Proceedings of the 2025 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2025, pp. 84–98. [Online]. Available: <https://doi.org/10.1109/IISWC66894.2025.00017>
- [21] I. Gim, C. Li, and L. Zhong, “Confidential prompting: Protecting user prompts from cloud LLM providers,” *arXiv preprint arXiv:2409.19134*, 2024. [Online]. Available: <https://arxiv.org/abs/2409.19134>

- [22] T. Bodea, M. Misono, J. Pritzi, P. Sabanic, T. Sommer, H. Unnibhavi, D. Schall, N. Santos, D. Stavrakakis, and P. Bhatotia, “Trusted AI agents in the cloud,” *arXiv preprint arXiv:2512.05951*, 2025. [Online]. Available: <https://arxiv.org/abs/2512.05951>
- [23] J. H. Saltzer and M. D. Schroeder, “The protection of information in computer systems,” *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.
- [24] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero trust architecture,” *NIST special publication*, vol. 800, no. 207, pp. 1–52, 2020.
- [25] D. Petrocelli and J. M. Fernández, “A defense-in-depth architecture for protecting AI agent intellectual property in multi-tenant cloud environments,” in *Cloud Computing, Big Data & Emerging Topics (JCC-BD&ET 2026)*, ser. Communications in Computer and Information Science (CCIS). Springer, 2026, tODO: volume, pages, DOI.
- [26] K.-H. Hung, C.-Y. Ko, A. Rawat, I.-H. Chung, W. H. Hsu, and P.-Y. Chen, “Attention tracker: Detecting prompt injection attacks in LLMs,” in *Findings of ACL: NAACL 2025*, 2025, pp. 2309–2322.
- [27] S. Thornton, “TRYLOCK: Defense-in-depth against LLM jailbreaks via layered preference and representation engineering,” *arXiv preprint arXiv:2601.03300*, 2026. [Online]. Available: <https://arxiv.org/abs/2601.03300>
- [28] D. Schwarz, “Countermind: A multi-layered security architecture for large language models,” *arXiv preprint arXiv:2510.11837*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.11837>
- [29] C. Pathade, V. Dhimam, S. Ahmad, and I. Lareb, “Serverless AI security: Attack surface analysis and runtime protection mechanisms for FaaS-based machine learning,” *arXiv preprint arXiv:2601.11664*, 2026. [Online]. Available: <https://arxiv.org/abs/2601.11664>
- [30] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, “A watermark for large language models,” in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- [31] G. K. R. Lau, X. Niu, H. Dao, J. Chen, C.-S. Foo, and B. K. H. Low, “Waterfall: Framework for robust and scalable text watermarking and provenance for LLMs,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 20 432–20 466.
- [32] S. Dathathri, A. See, S. Ghaisas, P.-S. Huang, R. McAdam, J. Welbl *et al.*, “Scalable watermarking for identifying large language model outputs,” *Nature*, vol. 634, pp. 818–823, 2024.
- [33] H. M. Bashir, Q. Li, and J. Hou, “A high capacity text steganography utilizing unicode zero-width characters,” in *2020 International conferences on internet of things (iThings) and IEEE green computing and communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData) and IEEE Congress on Cybermatics (Cybermatics)*. IEEE, 2020, pp. 668–675.
- [34] Y. Mathew, O. Matthews, R. McCarthy, J. Velja, C. Schroeder de Witt, D. Cope, and N. Schoots, “Hidden in plain text: Emergence and mitigation of steganographic collusion in LLMs,” in *Proceedings of IJCNLP-AACL*, 2025.
- [35] National Institute of Standards and Technology, “Adversarial machine learning: A taxonomy and terminology of attacks and mitigations,” NIST, Tech. Rep. AI 100-2e, 2025.

Citation: D. Petrocelli and J. M. Fernández. *Layered Prompt Protection for AI Agents in Customer-Controlled Cloud Accounts: Model Robustness and Defense-Layer Contribution*. Journal of Computer Science & Technology, vol. XX, no. X, pp. X–X, 2026.
DOI: to be assigned at the copyediting stage.
Received: to be completed. **Accepted:** to be completed.
Copyright: This article is distributed under the terms of the Creative Commons License CC-BY-NC-SA.